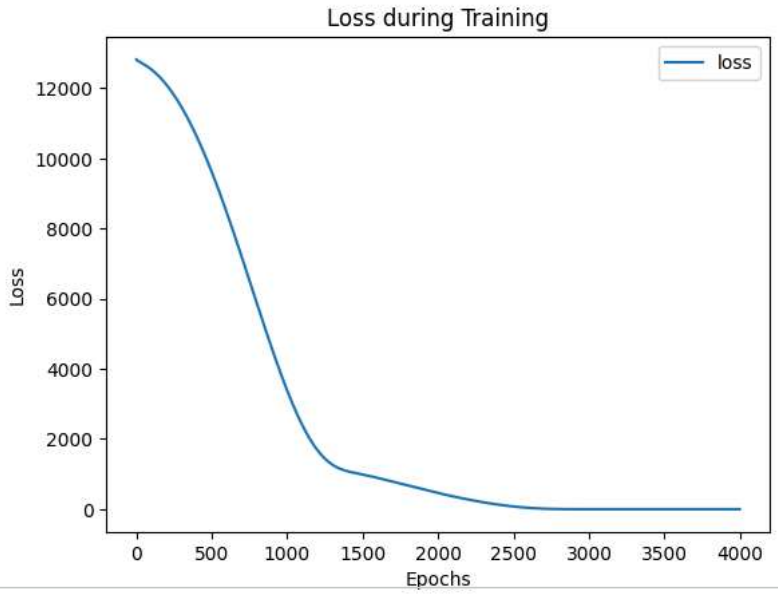
```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

dataset1 = pd.read_excel('/content/sample_data/dldata.xlsx')
X = dataset1[['Input']].values
y = dataset1[['Output']].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=33)


scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)


class NeuralNet(nn.Module):
  def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(1, 8)
        self.fc2 = nn.Linear(8, 10)
        self.fc3 = nn.Linear(10, 1)
        self.relu = nn.ReLU()
        self.history = {'loss': []}

  def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

ai_teja= NeuralNet ()
criterion = nn. MSELoss ()
optimizer = optim.RMSprop (ai_teja. parameters(), lr=0.001)

def train_model(ai_teja, X_train, y_train, criterion, optimizer, epochs=4000) :
  for epoch in range (epochs) :
    optimizer. zero_grad()
    loss = criterion(ai_teja(X_train), y_train)
    loss. backward()
    optimizer.step()
    ai_teja. history['loss'] .append(loss.item())
    if epoch % 200 == 0:
      print(f'Epoch [{epoch}/{epochs}], Loss: {loss.item():.6f}')

train_model(ai_teja, X_train_tensor, y_train_tensor, criterion, optimizer)



with torch.no_grad():
    test_loss = criterion(ai_teja(X_test_tensor), y_test_tensor)
    print(f'Test Loss: {test_loss.item():.6f}')



loss_df = pd.DataFrame(ai_teja.history)


import matplotlib.pyplot as plt
loss_df.plot()
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss during Training")
plt.show()


X_n1_1 = torch.tensor([[9]], dtype=torch.float32)
prediction = ai_teja(torch.tensor(scaler.transform(X_n1_1), dtype=torch.float32)).item()
print(f'Prediction: {prediction}')
```

```
Epoch [0/4000], Loss: 12818.516602
Epoch [200/4000], Loss: 12093.368164
Epoch [400/4000], Loss: 10621.029297
Epoch [600/4000], Loss: 8442.650391
Epoch [800/4000], Loss: 5850.138672
Epoch [1000/4000], Loss: 3376.558105
Epoch [1200/4000], Loss: 1683.569214
Epoch [1400/4000], Loss: 1081.863159
Epoch [1600/4000], Loss: 891.170776
Epoch [1800/4000], Loss: 673.176453
Epoch [2000/4000], Loss: 461.087921
Epoch [2200/4000], Loss: 275.056580
Epoch [2400/4000], Loss: 130.169067
Epoch [2600/4000], Loss: 38.526253
Epoch [2800/4000], Loss: 3.418624
Epoch [3000/4000], Loss: 0.006783
Epoch [3200/4000], Loss: 0.016483
Epoch [3400/4000], Loss: 0.006384
Epoch [3600/4000], Loss: 0.009862
Epoch [3800/4000], Loss: 0.008907
Test Loss: 1.003619
```




```
Prediction: 90.08489227294922
```